

**VNUHCM - UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY**



Final Semester Project
Transformer-based NLP Application:
Hate Speech Classifier

Lecturer:

- Ngô Minh Nhựt
- Lê Long Quốc

Class: Học thống kê - 21CNTThuc

Group member:

- 21127205 – Cao Hoài Yến Vy
- 21127412 – Hồ Bạch Như Quỳnh
- 22127096 – Trần Phúc Hải

Content

1. Introduction	1
2. Dataset	1
2.1. About the dataset (HateXplain)	1
2.2. Data preprocessing	2
2.3. Data split (Train/Validation)	3
3. Model operation, fine-tuning techniques, training process (Comprehensive overview)	4
3.1. Model overview	5
3.2. Architectural Framework	5
3.3. Training process	7
3.3.1. Training Process with BERT Masked Language Model (MLM)	8
3.3.2. Training Process with BERT for Sequence Classification	9
4. Evaluate the model	11
4.1. Metric used	11
4.2. Evaluate result	11
5. Web-based application	13
6. References	17

1. Introduction

In the digital era, the proliferation of online platforms has brought about both incredible opportunities for communication and significant challenges in maintaining a safe and respectful environment. One of the most pressing issues faced by these platforms is the presence of hate speech, which can manifest in various forms, including comments, posts, and messages. Hate speech is generally understood as any form of communication that disparages, discriminates against, or incites violence toward individuals or groups based on characteristics such as race, religion, ethnicity, gender, or sexual orientation. The impact of such harmful language extends beyond the virtual world, contributing to social division, emotional distress, and, in some cases, real-world violence.

In this project, we plan to develop a language model based on the Transformer architecture to address this issue, the proposed solution focuses on the detection and classification of hate speech in input sentences, categorizing them into one of three distinct categories: hateful, offensive, or normal. Each of these categories serves a crucial role in understanding the nature of the content and determining the appropriate response. "Hateful" content is the most severe, often explicitly targeting and inciting harm against specific groups. "Offensive" content, while not necessarily inciting violence, can still perpetuate negative stereotypes or use derogatory language. Finally, "normal" content is free of harmful language and contributes to healthy discourse.

2. Dataset

2.1. About the dataset (HateXplain)

- Link to the dataset: <https://www.kaggle.com/datasets/kiitand/hatexplain-dataset>
- HateXplain is the first benchmark hate speech dataset covering multiple aspects of the issue. Each post in the dataset is annotated from three different perspectives:
 - The basic, commonly used 3-class classification: hatespeech, offensive or normal
 - The target community: the community that has been the victim of hate speech/offensive speech in the post. For the target community section, only the most frequently occurring categories are included. These categories are: African, Islam, Jewish, Homosexual, Women, Refugee, Arab, Caucasian, Hispanic, and Asian.
 - The rationales: refers to the important tokens in the post (labeled 1) that each annotator uses to justify their labeling decisions.

- The dataset includes 20148 samples having structure similar to below:

```
{
  "id": "24198545_gab",
  "annotators": [
    {
      "label": "hatespeech",
      "annotator_id": 4,
      "target": ["African"]
    },
    {
      "label": "hatespeech",
      "annotator_id": 3,
      "target": ["African"]
    },
    {
      "label": "offensive",
      "annotator_id": 5,
      "target": ["African"]
    }
  ],
  "rationales": [[0,0,0,0,0,0,0,0,0,1,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]],
  "post_tokens": ["and", "this", "is", "why", "i", "end", "up", "with", "nigger", "trainee", "doctors", "who",
    "can", "not", "speak", "properly", "lack", "basic", "knowledge", "of", "biology", "it",
    "truly", "scary", "if", "the", "public", "only", "knew"]
}
```

A sample in HateXplain dataset

- Data fields:
 - id (post_id): unique id for each post.
 - annotators: the list of annotations from each annotator.
 - annotators[label]: the label assigned by the annotator to this post (hatespeech, normal, offensive).
 - annotators[annotator_id]: the unique id assigned to each annotator.
 - annotators[target]: list of targets that the annotator thinks are being attacked.
 - rationales: list of rationales selected by annotators. Each rationale represents a list with values 0 or 1. A value of 1 means that the token is part of the rationale selected by the annotator. Essentially, the annotator thinks this token is crucial in identifying the harmful content. It is 0 otherwise. To get the particular token, we can use the same index position in "post_tokens".
 - post_tokens: list of tokens representing the post which was annotated.

2.2. Data preprocessing

- For each post, the labels assigned by the annotators are gathered, and the most frequent label is determined. If this label appears more than 50% of the time, it is assigned as the final label. Posts where no label reaches this threshold are excluded from the dataset, which ultimately results in a refined dataset of 19,229 posts.

- Similarly, the target communities are determined by analyzing the annotators' inputs. The majority target communities, if any, are identified and appended to the final target list for each post.
- The rationales provided by the annotators are also processed. The rationales are averaged across all annotators to give an overall view of which parts of the post are considered relevant. These averaged values are then converted into binary labels, where values greater than 0.5 are marked as important (1), and those below 0.5 are not (0). This binary format simplifies the identification of key parts of the post. If no rationales are available, the entire post is labeled with zeros.
- Additionally, the tokens in each post are combined into a single text string for easier handling in future steps for text classification, ensuring that each post is represented as a unified body of text rather than individual tokens. The post ID is also recorded for future reference.
 - Example: ["This", "is", "an", "example", "of", "a", "social", "media", "post"] becomes "This is an example of a social media post".
- Cleans the data by removing unnecessary HTML tags.
- Filters and processes the target community information from the original data. It identifies the communities of interest and retains these if they are present in the post. If no relevant communities are found, it assigns 'None'. The filtered results are then stored back into the "**Final_target**" column, and a list of unique communities present in the data is extracted.
- Finally, all the data is organized and stored in a DataFrame, the distribution of each label is as follows
 - normal: 7814 samples
 - hatespeech: 5935 samples
 - offensive: 5480 samples

	post_ids	input_text	rationales	label	Final_target
0	1179055004553900032_twitter	i dont think im getting my baby them white 9 h...	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	normal	[None]
1	1179063826874032128_twitter	we cannot continue calling ourselves feminists...	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	normal	[None]
2	1178793830532956161_twitter	nawt yall niggers ignoring me	[0, 0, 0, 0, 0]	normal	[African]
3	1179088797964763136_twitter	i am bit confused coz chinese ppl can not acce...	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, ...	hatespeech	[Asian]
4	1179085312976445440_twitter	this bitch in whataburger eating a burger with...	[0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	hatespeech	[Caucasian, Women]

Dataset after preprocessing

2.3. Data split (Train/Validation)

The dataset is split into two parts: **80% for training and 20% for validation**. This split is commonly used in machine learning tasks to ensure that the model has sufficient data to learn patterns during training while also setting aside a portion of the data for validation. The training set is used to train the model, allowing it to learn patterns and adjust its parameters, while the validation set is used to monitor the model's performance on unseen data.

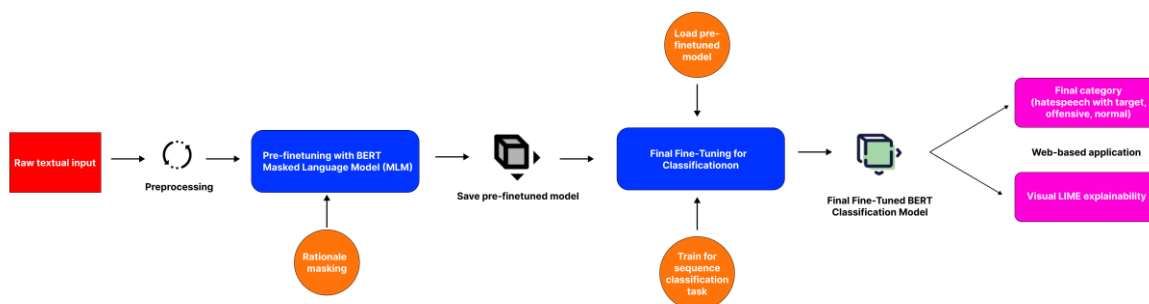
The choice of an 80/20 split balances two factors:

- Training size: By using 80% of the data for training, we ensure that the model has access to a large amount of data, which helps it learn effectively.
- Validation set size: The remaining 20% is sufficient to evaluate the model's performance on data it hasn't seen during training, providing a good indicator of how well the model generalizes to new examples.

This split is performed using **stratified sampling** to ensure that the class distribution (normal, offensive, and hateful) remains similar in both the training and validation sets, which helps maintain a balanced dataset for evaluation.

3. ***Model operation, fine-tuning techniques, training process (Comprehensive overview)***

This framework employs a **two-phase** approach to enhance text classification using a BERT-based model. Initially, the model is pre-finetuned with a Masked Language Model (MLM) to learn domain-specific patterns through rationale masking, where important tokens are selectively masked. The pre-finetuned model is then saved and later used for fine-tuning on a sequence classification task. This final phase involves training the model to categorize input texts into predefined classes, leveraging the domain knowledge gained during pre-finetuning to improve classification accuracy.



Overall Flow of the Application

What is the final finetuning step for?

The final finetuning step is where the pre-finetuned model, which has already learned domain-specific knowledge, is further trained on a sequence classification task. The model uses its pre-finetuned weights as a starting point and then learns how to classify inputs into one of three categories. This step leverages the domain-specific understanding gained during pre-finetuning to improve classification accuracy.

3.1. Model overview

The model employed in this study is **BertForMaskedLM (BERT for Masked Language Modeling)**, sourced from the Hugging Face Transformers library.

Why is the model pre-finetuned using BERT MLM with rationale information?

The pre-finetuning step using BERT's Masked Language Model (MLM) with rationale information is designed to make the model aware of domain-specific knowledge. By masking and predicting parts of the text where the rationale is '1', the model is encouraged to focus on the most relevant parts of the text. This helps the model understand which tokens are important in the context of the task, thereby enhancing its ability to handle the specific type of data it will encounter in the classification phase.

3.2. Architectural Framework

The BertForMaskedLM model is a sophisticated deep learning model designed to perform Masked Language Modeling (MLM). It is built on the BERT (Bidirectional Encoder Representations from Transformers) architecture, which consists of several key components, each playing a crucial role in processing and understanding textual data. The following sections outline the key architectural components and their roles in processing textual data, with a particular emphasis on the implementation details related to rationale masking.

1. Token Embedding Layer:

Functionality: The token embedding layer converts each token in the input sequence into a dense vector representation, capturing semantic nuances. This layer integrates three types of embeddings: token embeddings, segment embeddings, and positional embeddings. These embeddings collectively encode the identity of each token, the segment it belongs to, and its position within the sequence.

Application in the Context of This Model: The tokenization process involves converting text into token IDs using a predefined tokenizer, which is then mapped to embeddings that are fed into the model.

2. Positional Encoding:

Functionality: Transformers, being inherently permutation-invariant, require positional encodings to maintain the sequential order of tokens. Positional encodings are added to the token embeddings, providing the model with information regarding the order of words within the sequence.

Application in the Context of This Model: The positional encodings are implicitly handled as part of the tokenization process, ensuring that the sequential relationships among tokens are preserved during processing.

3. Self-Attention mechanism:

Functionality: The self-attention mechanism is central to the BERT architecture. It computes the relevance of each token relative to all other tokens in the sequence, allowing the model to focus on contextually important tokens. This mechanism enables the model to capture long-range dependencies and intricate relationships within the text.

Application in the Context of This Model: The rationale-based masking approach, where specific tokens are masked based on their importance (as indicated by the rationale), enhances the effectiveness of the self-attention mechanism by emphasizing contextually significant tokens.

4. Transformer encoder blocks:

Functionality: The model is composed of multiple stacked transformer encoder blocks, each consisting of a multi-head self-attention layer followed by a feed-forward neural network. These encoder blocks progressively refine the representations of tokens, capturing higher-level semantic information with each layer.

Application in the Context of This Model: After tokenization and masking, the sequence is processed through these encoder blocks, which iteratively refine the token embeddings and produce contextually enriched representations.

5. Masked Language Modeling (MLM) Head:

Functionality: The MLM head is the specialized output layer designed for the task of predicting the original tokens that were masked during the input processing phase. It generates a probability distribution over the entire vocabulary for each masked token, enabling accurate prediction based on the surrounding context.

Application in the Context of This Model: The implementation includes a rationale-based masking strategy, where selected tokens are replaced with a mask token. The MLM head then utilizes the contextual information processed by the encoder blocks to predict the masked tokens, aligned with the rationale annotations.

6. Fine-Tuning Layer:

Functionality: In downstream tasks, the model can be further fine-tuned by adding task-specific layers. This fine-tuning allows the model to adapt its pre-learned representations for specific objectives such as text classification or question answering.

Relevance to Implementation: Following pre-finetuning with the MLM task, the model can be fine-tuned for sequence classification. The pre-trained weights provide a solid foundation, incorporating domain-specific knowledge learned during the MLM phase.

3.3. Training process

This framework utilizes a two-phase approach to enhance text classification with a BERT-based model. The model chosen for this study is BertForMaskedLM from the Hugging Face Transformers library.

Phase 1: Pre-Finetuning with BERT MLM

In the initial phase, the model undergoes pre-finetuning using BERT's Masked Language Model (MLM). This step involves training the model on domain-specific data, with a particular focus on rationale masking. Important tokens in the text, as identified by the rationale (where rationale is marked as '1'), are selectively masked. The objective of this pre-finetuning phase is to:

- **Impart Domain Knowledge:** By masking tokens that are deemed important, the model learns to predict these masked tokens based on their context. This process helps the model acquire an understanding of domain-specific patterns and nuances, crucial for handling the specific data in subsequent tasks.

Phase 2: Final Fine-Tuning for Sequence Classification

Following the pre-finetuning phase, the model is saved and later used for the final fine-tuning phase. In this phase, the pre-finetuned model is further trained on a sequence classification task. The model's weights, which have been informed by the domain-specific knowledge from the pre-finetuning phase, serve as a starting point. During final fine-tuning:

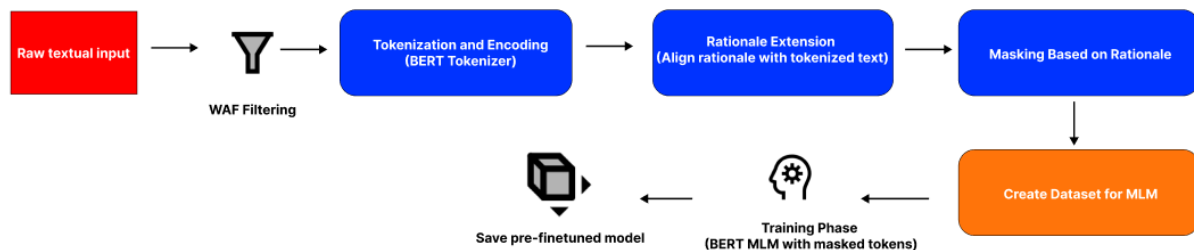
- **Sequence Classification:** The model is trained to categorize input texts into predefined classes. Leveraging the domain-specific understanding gained from the pre-finetuning phase, the model refines its ability to accurately classify texts into one of the specified categories.

Rationale for Pre-Finetuning with BERT MLM

The use of BERT MLM with rationale information for pre-finetuning is crucial for enhancing the model's performance on domain-specific data. This approach:

- **Focuses the Model on relevant tokens:** By masking and predicting critical parts of the text, the model is encouraged to concentrate on tokens that are significant within the context of the task.
- **Enhances Task-Specific understanding:** The domain-specific knowledge acquired during pre-finetuning improves the model's ability to handle and classify data accurately in the final phase.

3.3.1. Training Process with BERT Masked Language Model (MLM)



Process 1: BERT Masked Language Model (MLM) Pre-Finetuning

3.3.1.1. Model and Dataset

Model: The training process utilizes the BertForMaskedLM model from the Hugging Face Transformers library. This model is designed for pre-training using a Masked Language Model (MLM) objective.

Dataset: The dataset consists of text data with associated rationales. Each text entry is paired with a rationale indicating the importance of specific tokens.

3.3.1.2. Data Preparation

Tokenization: Texts are tokenized using the BertTokenizer. Tokenization converts raw text into tokens that the model can process.

Rationale Extension: Rationale labels (binary values indicating importance) are extended to match the tokenized text, considering subword tokenization.

Masking: A certain percentage of tokens, marked as important by the rationale, are randomly selected for masking. This helps the model learn to predict masked tokens based on the surrounding context.

Padding: Input sequences are padded to a maximum length of 180 tokens, and labels are adjusted accordingly with -100 for padding tokens.

3.3.1.3. Training Setup

Model Initialization: The BertForMaskedLM model is initialized using pre-trained weights from bert-base-uncased.

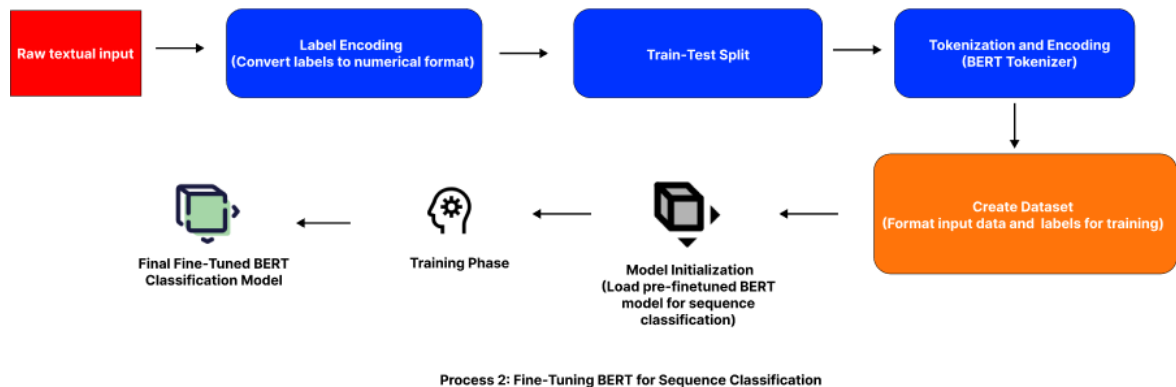
Device: The model is moved to a GPU if available to leverage hardware acceleration.

3.3.1.4. Training Process

Data Loader: A custom TextDataset class is defined to handle data loading and preprocessing for the MLM task.

Training Loop: During training, the model processes batches of tokenized text with masked tokens. The loss is computed, gradients are updated, and the model parameters are optimized.

3.3.2. Training Process with BERT for Sequence Classification



3.3.2.1. Model and Dataset

Model: The BertForSequenceClassification model is employed, which is fine-tuned for sequence classification tasks. This model is loaded with weights from a previously fine-tuned BERT model from the previous process.

Dataset: The dataset for classification includes input text and labels for multi-class classification. Labels are encoded into numerical format.

3.3.2.2. Data Preparation

Label Encoding: Labels are converted into a numerical format using LabelEncoder to facilitate model training.

Train-Test Split: The data is split into training (80%) and validation (20%) sets like mentioned in 2.3, using train_test_split from scikit-learn. This split ensures that the model is trained and validated on separate subsets of the data.

3.3.2.3. Tokenization and Formatting

Tokenization: The BertTokenizer is used to tokenize the input texts, handling padding and truncation to ensure uniform input size.

Dataset Creation: Tokenized inputs and attention masks are combined with labels to create TensorDataset objects for training and validation.

3.3.2.4. Training Setup

DataLoader: Data is loaded in batches using DataLoader for both training and validation sets.

Device: The model is moved to a GPU if available.

3.3.2.5. Training Process

Training Loop: The training loop involves:

- Moving data batches to the appropriate device.
- Performing forward passes to compute loss.
- Backward passes to compute gradients.
- Optimizing the model parameters.
- Tracking and printing the average training loss per epoch.
- Early stopping mechanism (Explained below)

Regularization:

- **Optimizer and Loss Function:** The AdamW optimizer (Adam with Weight Decay) is used during training, which incorporates **weight decay** as a regularization technique. Weight decay helps to reduce overfitting by adding a penalty to the loss function for large weights, encouraging the model to learn smaller, more generalized weights. In this project, a weight decay of 0.01 is applied to all model parameters, helping the model generalize better and reduce overfitting.
- **Dropout:** BERT models already include **dropout layers** in their architecture, which provide an additional method to reduce overfitting. Dropout works by randomly setting a fraction of the input units to zero during training, preventing the model from becoming overly reliant on any one feature. This built-in dropout mechanism enhances the model's ability to generalize on unseen data without requiring additional implementation.
- These regularization techniques, combined with **early stopping**, ensure that the model avoids overfitting and performs optimally on unseen validation data.

Early Stopping Mechanism

To prevent overfitting during the training process, we implement an early stopping mechanism. This technique monitors the model's performance on the validation set after

each training epoch and halts the training if the validation loss does not improve significantly over a set number of consecutive epochs (referred to as "patience"). Specifically, the early stopping mechanism is triggered if the validation loss does not improve by at least a predefined threshold (called min_delta) for patience number of epochs.

By employing early stopping, the model is trained only for as long as necessary to achieve optimal performance, avoiding overfitting to the training data. This not only speeds up training time but also helps the model generalize better on unseen data. The best-performing model, based on validation loss, is saved during training and restored after early stopping is triggered, ensuring that the final model used for evaluation is the one with the best validation performance.

In this project, the early stopping mechanism is configured with the following parameters:

- Patience: 3 epochs
- Min Delta: 0.001 improvement required to reset the patience counter.

4. *Evaluate the model*

4.1. Metric used

- Accuracy: a basic and intuitive metric, especially useful when the data is evenly distributed among classes. It indicates the proportion of correct predictions made by the model.
- Precision: important in imbalanced classification problems, particularly when the cost of false positives is high. Precision tells you how accurate the model is when predicting a specific class.
- Recall: provides information about the model's ability to identify all relevant instances of a class. Recall is crucial when missing instances of a particular class has significant consequences.
- F1 Score: the F1 Score is a good overall metric when you need to balance Precision and Recall, especially in cases of imbalanced classes where you want to avoid being overly influenced by one metric.
- AUC-ROC (Area Under the Curve - Receiver Operating Characteristic): AUC-ROC provides insight into the model's ability to distinguish between classes across different thresholds, which is particularly important in applications where the balance between true positive and false positive rates is crucial.

4.2. Evaluate result

- Result:

AUROC: 0.858129956052606				
	precision	recall	f1-score	support
hatespeech	0.730	0.830	0.777	1187
normal	0.730	0.758	0.744	1563
offensive	0.600	0.478	0.532	1096
accuracy			0.700	3846
macro avg	0.687	0.689	0.684	3846
weighted avg	0.693	0.700	0.694	3846

- Comment

- The model achieves an overall accuracy of 70%, which indicates that it correctly classifies 7 out of every 10 instances.
- High AUROC Score (0.8581) signifies that the model has a strong capability to distinguish between the classes. An AUROC close to 1 suggests that the model is effective in ranking positive instances higher than negative ones across all classes.
- The model shows a high recall (0.830) for the "hatespeech" category, meaning it successfully identifies a large proportion of actual hate speech instances. This is critical in scenarios where missing hateful content could have severe negative consequences.
- Both precision and recall around 0.73-0.76 for the "normal" class, the model reliably identifies non-offensive content without excessive false positives or negatives. This balance ensures that regular communication remains unflagged, maintaining user trust and system usability.
- The model's performance on the "offensive" class is notably weaker, with a precision of 0.600 and a recall of 0.478. This indicates that the model misses more than half of the offensive instances and incorrectly labels a significant number of non-offensive instances as offensive.

- Compare with other models:

Model	AUROC	Accuracy	Macro F1
BiRNN-HateXplain [Attn]	0.805	0.629	0.629

BiRNN-HateXplain [LIME]	0.843	0.629	0.629
BERT-HateXplain [Attn]	0.851	0.698	0.687
BERT-HateXplain [LIME]	0.851	0.698	0.687
BertMLM + Bert Classifier (LIME)	0.858	0.700	0.684

- Our model has the highest AUROC of 0.858, showcasing its superior ability to discriminate between the different classes (normal, hate speech, offensive). This metric highlights that our model has the best performance in terms of distinguishing between classes compared to the other models.
- Our model achieves the highest accuracy at 0.700, meaning it correctly classifies the sentences more frequently than any of the other models. This indicates that our model is the most reliable in terms of overall correct predictions.
- The Macro F1 Score is 0.684, which is slightly lower than the 0.687 of the BERT-HateXplain models, it still performs competitively. This metric reflects how well your model balances precision and recall across all classes.

5. *Web-based application*

This web application provides an interactive interface for classifying text as hate speech or non-hate speech using a fine-tuned BERT model. The classification process is enhanced with LIME (Local Interpretable Model-agnostic Explanations), which offers visual insights into the model's decision-making process.

Features:

- Input Text Classification:
 - Users can input any text into a text area provided by the interface.
 - Upon submission, the text is processed and classified using a pre-finetuned BERT model trained specifically for detecting hate speech.
- Prediction and explanation:
 - The application displays the predicted label (e.g., "hate speech" or "normal") based on the input text.
 - If the text is classified, the app uses LIME to generate an explanation, highlighting the words in the text that contributed most to the classification. This is presented in a visual, interactive format, making it easy to understand the model's reasoning.

- **Target Identification:** In cases where the text is classified as hate speech or offensive, the app goes further by identifying the target group of the hate speech (e.g., specific ethnic or social groups).
- **Interactive visualization:** The explanation generated by LIME is embedded directly into the web interface as an HTML component, allowing users to see a detailed breakdown of which words or phrases influenced the model's decision.

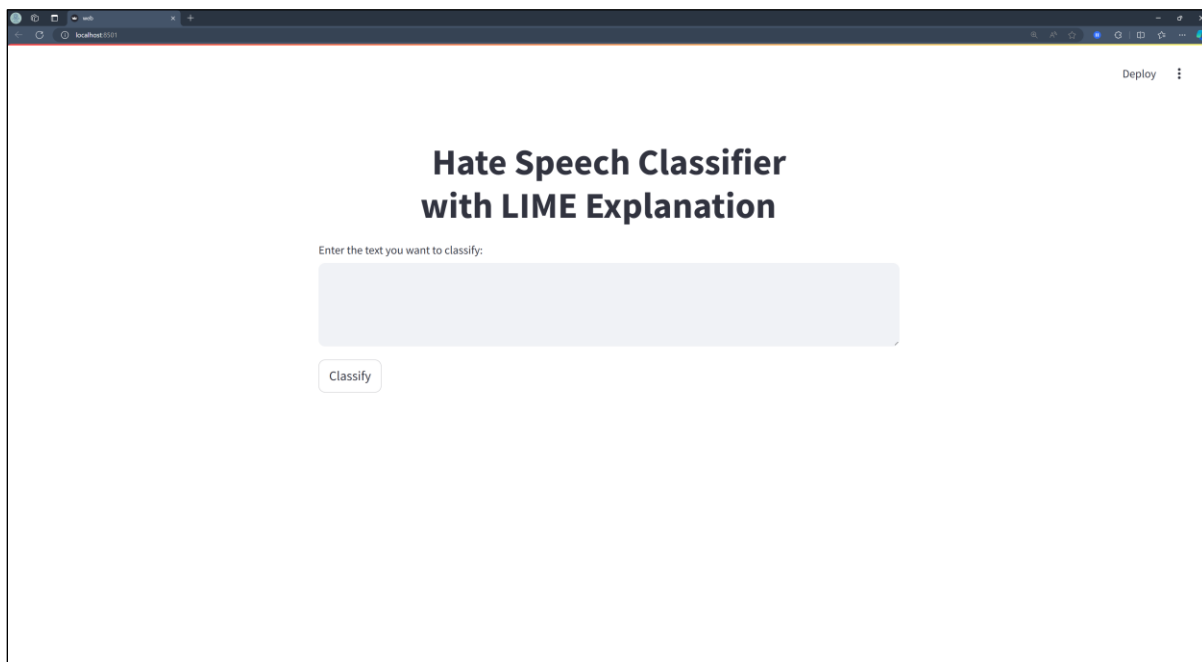
Technical overview:

- The application is built using Streamlit.
- The backend utilizes a fine-tuned BERT model for text classification, sourced from a pre-trained model and further fine-tuned on a specific hate speech dataset.
- The model and tokenizer are loaded directly from the pre-trained model files, ensuring consistency in predictions.

For target group identification, the application employs Sentence-BERT to encode the input text and compute similarities with a dataset of pre-labeled instances.

Web UI

This is a full view of the website layout. Zoomed-in example usage is below.



Some examples

- Hate speech

Hate Speech Classifier with LIME Explanation

Enter the text you want to classify:

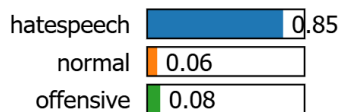
hello what the heck are you doing, you hippy hypocrite nigger?

Classify

Predicted Label: hatespeech

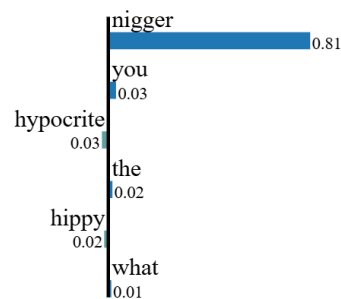
Target of Hate Speech: ['African', 'Homosexual']

Prediction probabilities



NOT hatespeech

hatespeech



Text with highlighted words

hello what the heck are you doing, you hippy hypocrite nigger?

- Normal

Hate Speech Classifier with LIME Explanation

Enter the text you want to classify:

It is a wonderful day, isn't it? Such a great time to go play badminton.

Classify

Predicted Label: normal

Prediction probabilities

hatespeech	0.01
normal	0.89
offensive	0.10

NOT normal

normal

badminton	0.02
a	0.01
play	0.01
to	0.01
t	0.01
it	0.01
	0.01

Text with highlighted words

It is a wonderful day, isn't it? Such a great time to go play badminton.

- Offensive

Hate Speech Classifier with LIME Explanation

Enter the text you want to classify:

that bitch stole my husband, she is a whore!

Classify

Predicted Label: offensive

Target of Hate Speech: ['Women', 'Islam']

Prediction probabilities

hatespeech	0.08
normal	0.15
offensive	0.77

NOT offensive

offensive

Text with highlighted words

that bitch stole my husband, she is a whore!

Feature importance for 'offensive' prediction:

Feature	Importance
bitch	0.46
whore	0.12
that	0.07
stole	0.04
husband	0.04
is	0.03

6. References

- R. Mathew et al., "HateXplain: A Benchmark Dataset for Explainable Hate Speech Detection,"
<https://arxiv.org/pdf/2012.10289>
- J. Kim, B. Lee, and K.-A. Sohn, "Why Is It Hate Speech? Masked Rationale Prediction for Explainable Hate Speech Detection,"
<https://arxiv.org/pdf/2211.00243v1>

- Papers with code - HateXplain: A benchmark dataset for explainable hate speech detection HateXplain: A Benchmark Dataset for Explainable Hate Speech Detection | Papers With Code. Available at: <https://paperswithcode.com/paper/hatexplain-a-benchmark-dataset-for> (Accessed: 04 September 2024).
- Harshmeetsingh Chandhok, "Stopping The Hate," [Stopping The Hate | Devpost](#)